

RUNNING THE BORDER SURVEILLANCE SYSTEM

1. Clone the Repository

```
git clone https://github.com/your-username/Aegis-X.git  
cd Aegis-X
```

2. Create a Virtual Environment

Windows

```
python -m venv venv  
venv\Scripts\activate
```

Linux / macOS

```
python3 -m venv venv  
source venv/bin/activate
```

3. Install Required Dependencies

```
pip install -r requirements.txt
```

If **requirements.txt** is not available:

```
pip install ultralytics  
pip install opencv-python  
pip install fastapi  
pip install uvicorn  
pip install langgraph  
pip install groq  
pip install python-jose  
pip install python-dotenv  
pip install google-genai  
pip install langchain-core
```

4. Configure Environment Variables

Create a `.env` file in the project root directory.

```
GOOGLE_API_KEY=your_google_api_key
TAVILY_API_KEY=your_tavily_api_key
GROQ_API_KEY=your_groq_api_key
```

5. Download the Trained YOLOv8 Model

Place the trained model file inside:

```
models/
└── aegis_yolo_v8.pt
```

6. Verify Project Structure

```
Aegis-X/
├── models/
│   └── aegis_yolo_v8.pt
├── backend/
│   ├── inference.py
│   ├── api.py
│   ├── security.py
│   └── config.py
├── frontend/
├── .env
├── requirements.txt
└── README.md
```

7. Start the FastAPI Server

```
uvicorn api:app --reload
```

Expected Output:

INFO: Uvicorn running on http://127.0.0.1:8000

8. Open API Documentation

Navigate to:

<http://127.0.0.1:8000/docs>

FastAPI automatically generates interactive API documentation.

9. Start the Real-Time Intelligence Stream

Open a browser and visit:

<http://127.0.0.1:8000/v1/stream/intelligence>

Example Output:

```
{  
  "agent": "Supervisor",  
  "thought": "Weapon detected at Sector 4. Initiating deep scan...",  
  "severity": "CRITICAL"  
}
```

10. Run the Surveillance Detection Module

```
python inference.py
```

The system will:

- Load the custom YOLOv8 model
- Analyze incoming camera frames

- Detect weapons and suspicious objects
 - Generate threat alerts
 - Forward detections to the AI Agent Framework
-

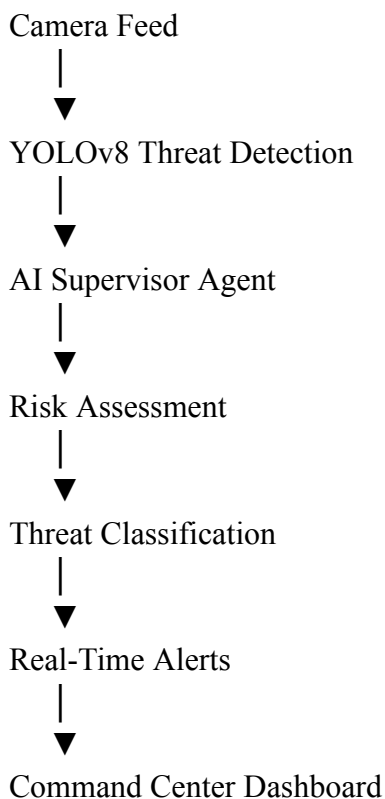
11. Launch the Command Center Dashboard

`python dashboard.py`

The dashboard displays:

- Live camera feed
 - Intruder detection bounding boxes
 - Threat severity levels
 - Sector information
 - AI-generated threat assessments
 - Historical threat logs
-

12. System Workflow



Expected Output

Normal Monitoring State

System Online
No Active Threats
Threats Today: 0
Critical Alerts: 0

Threat Detected State

Weapon Detected
Sector: 4
Severity: CRITICAL
Status: Intruder Detected
Alert Generated Successfully

Stop the Application

Press:

CTRL + C

to terminate the FastAPI server and all running surveillance services.